

Unit 1: Language Preliminaries — Detailed Notes

1. Introduction to .NET Framework

Description

The .NET Framework is a Microsoft software platform for building and running Windows applications. It provides a managed execution environment (CLR) and a rich class library (FCL) so developers don't need to write low-level code for memory management, security, or type-checking.

Syntax

Not a code construct itself — it is a platform. Represented conceptually as layers:

Application (C#, VB.NET, F#)



Framework Class Library (FCL)



Common Language Runtime (CLR)



Operating System

Types (Components)

Component	Role
CLR (Common Language Runtime)	Executes code, manages memory (GC), type safety
FCL (Framework Class Library)	Pre-built classes (System.IO, System.Collections, etc.)
CTS (Common Type System)	Defines rules for data types across languages
CLS (Common Language Specification)	Ensures cross-language compatibility

Use

- Build desktop, web, and enterprise applications
- Provides automatic memory management and security
- Enables multiple languages to interoperate (C#, VB.NET compiled code can call each other)

Full Example

using System;

```
namespace DotNetIntro
{
    class Program
    {
        static void Main(string[] args)
        {
            // Uses FCL class Console (from System namespace)
        }
    }
}
```

```

        Console.WriteLine("Hello from the .NET Framework!");
        Console.WriteLine("CLR Version: " + Environment.Version);
    }
}
}

```

Typical Assignment Program

Problem: Write a program that prints your name, the current date and time, and the CLR version being used, using only FCL classes.
using System;

```

class Assignment1
{
    static void Main()
    {
        Console.WriteLine("Name: John Doe");
        Console.WriteLine("Current Date & Time: " + DateTime.Now);
        Console.WriteLine("CLR Version: " + Environment.Version);
    }
}

```

2. Compilation and Execution of .NET Applications

Description

.NET applications go through a two-step compilation process: source code is compiled to **MSIL (Microsoft Intermediate Language)**, and at runtime the **JIT compiler** converts MSIL to native machine code for execution by the CLR.

Syntax (Command-line compilation example)

```

csc Program.cs    // compiles C# source into MSIL assembly (Program.exe)
Program.exe      // CLR loads assembly, JIT compiles MSIL to native code, executes

```

Types (JIT Compilers)

Type	Description
Pre-JIT	Compiles entire code to native code at once (at load time)
Econo-JIT	Compiles methods when needed, discards after use
Normal JIT	Compiles methods when called, caches for reuse during execution

Use

- Enables platform-neutral intermediate code (MSIL) that can run on any system with a compatible CLR
- Runtime optimization based on the actual machine/OS
- Type verification and security checks before execution

Full Example

```
// Program.cs
using System;

class CompileDemo
{
    static void Main()
    {
        Console.WriteLine("This source is compiled to MSIL, then JIT-compiled to native code.");
    }
}
```

Steps to compile & run manually:

csc CompileDemo.cs // Step 1: Source -> MSIL (CompileDemo.exe)

CompileDemo.exe // Step 2: CLR loads & JIT compiles -> Native code -> Runs

Typical Assignment Program

Problem: Explain with a program and diagram how a simple "Hello World" C# program moves from source code to execution, identifying each stage (compiler, MSIL, CLR, JIT).

```
using System;
```

```
class HelloWorldStages
{
    static void Main()
    {
        // Stage 1: This .cs file is source code
        // Stage 2: csc.exe compiles it into MSIL + metadata (HelloWorldStages.exe)
        // Stage 3: CLR loads the assembly at runtime
        // Stage 4: JIT compiler converts MSIL to native code
        // Stage 5: Native code executes on the CPU
        Console.WriteLine("Hello, World! (Demonstrating .NET Compilation Stages)");
    }
}
```

*To run a C# program in console, use the **.NET CLI**. Microsoft describes the **.NET CLI** as the command-line toolchain for developing, building, running, and publishing .NET apps.*

([Microsoft Learn](#))

1. Open console

Open one of these:

Command Prompt

PowerShell

Windows Terminal

VS Code Terminal

2. Create a new C# console project

`dotnet new console -n MyFirstProgram`

dotnet new creates a new project from a template, and console is the template for a console app. ([Microsoft Learn](#))

3. Go inside the project folder

`cd MyFirstProgram`

4. Open Program.cs

Replace the code with this:

using System;

```
class Program
{
    static void Main()
    {
        Console.WriteLine("Hello, World!");
    }
}
```

5. Run the program

`dotnet run`

dotnet run runs your application from the source code and builds it first if needed. ([Microsoft Learn](#))

Expected output

Hello, World!

You can also build only by using:

dotnet build

dotnet build compiles the project into output files. ([Microsoft Learn](#))

// Importing the System namespace

// It allows us to use Console.WriteLine()

Examples:

using System;

// Creating a class named Program

```
class Program
{
    // Main method is the starting point of the program
    static void Main()
    {
        // This line prints text on the screen
        Console.WriteLine("Hello, World!");
    }
}
```

Program to add two numbers with comments

```
// Importing the System namespace
using System;

// Creating a class named Program
class Program
{
    // Main method starts the program execution
    static void Main()
    {
        // Declaring first number
        int a = 10;

        // Declaring second number
        int b = 20;

        // Adding two numbers and storing result in sum
        int sum = a + b;

        // Displaying the result
        Console.WriteLine("The sum is: " + sum);
    }
}
```

Namespace:

Using a **namespace** in C# has these benefits:

1. **Organizes code**

It groups related classes together. Microsoft says a namespace puts types into an organized structure.

2. **Avoids name conflicts**

Two classes can have the same name if they are in different namespaces. Microsoft states that namespaces help prevent naming collisions.

3. **Makes big projects easier to manage**

For small programs, namespace is not very important. But in large projects, it helps separate code clearly.

using System;

```
// Creating a namespace
namespace CalculatorApp
{
    // Creating a class
```

```

class Program
{
    // Main method
    static void Main()
    {
        // Declaring two numbers
        int num1 = 10;
        int num2 = 20;

        // Adding numbers
        int sum = num1 + num2;

        // Displaying result
        Console.WriteLine("The sum is: " + sum);
    }
}

```

Syntax:

```

namespace SchoolApp
{
    class Student
    {
    }
}

```

```

namespace CollegeApp
{
    class Student
    {
    }
}

```

Example:

```

using System;

// First namespace
namespace SchoolApp
{
    class Student
    {
    }
}

```

```

        public void ShowInfo()
        {
            Console.WriteLine("This is a school student.");
        }
    }
}

```

```

// Second namespace
namespace CollegeApp
{
    class Student
    {
        public void ShowInfo()
        {
            Console.WriteLine("This is a college student.");
        }
    }
}

```

```

// Main program
class Program
{
    static void Main()
    {
        // Creating object of Student class from SchoolApp namespace
        SchoolApp.Student schoolStudent = new SchoolApp.Student();
        schoolStudent.ShowInfo();

        // Creating object of Student class from CollegeApp namespace
        CollegeApp.Student collegeStudent = new CollegeApp.Student();
        collegeStudent.ShowInfo();
    }
}

```

In C#, Main() can take command-line arguments using string[] args.

Example: static void Main(string[] args)

```
using System;
```

```

namespace ArgumentExample
{
    class Program

```

```

{
    // Main method with arguments
    static void Main(string[] args)
    {
        // args stores values passed from the console

        Console.WriteLine("Number of arguments: " + args.Length);

        // Display all arguments one by one
        for (int i = 0; i < args.Length; i++)
        {
            Console.WriteLine("Argument " + i + ": " + args[i]);
        }
    }
}

```

How to run with arguments

In console, run:

```
dotnet run Ram Shyam Hari
```

Output

Number of arguments: 3

Argument 0: Ram

Argument 1: Shyam

Argument 2: Hari

Simple meaning

```
static void Main(string[] args)
```

means:

args = values given by user when running the program

For example:

```
dotnet run Nepal Kathmandu
```

Then:

```
args[0] = Nepal
```

```
args[1] = Kathmandu
```

when we use internal class is used??

internal class is used when you want a class to be accessible only inside the same

project/assembly. Microsoft defines internal as an access modifier for types and members, and internal types or members can be accessed only within the same assembly.

Example

```
using System;
```

```

namespace MyApp
{
    // This class can be used only inside this same project/assembly
    internal class Student
    {
        public void ShowName()
        {
            Console.WriteLine("Student name is Ram");
        }
    }

    class Program
    {
        static void Main()
        {
            Student s = new Student();
            s.ShowName();
        }
    }
}

```

When do we use internal class?

Use internal class when the class is needed **inside your project**, but you do **not** want other projects to use it directly.

Example idea:

Project: SchoolManagement

internal class DatabaseHelper

DatabaseHelper may be useful inside the project, but other projects do not need to access it.

Difference between public and internal

```

public class Student
{
}

```

public means the class can be accessed from other projects too.

```

internal class Student
{
}

```

internal means the class can be accessed only inside the same project/assembly.

Also, if a top-level class has no access modifier, its default accessibility is internal, according to Microsoft's C# accessibility rules.

So this:

```
class Student
```

```
{  
}
```

is similar to:

```
internal class Student
```

```
{  
}
```

Simple meaning:

internal class = private for the project

inside namespace we must put internal class or not???

- **do not have to write internal class inside a namespace.**
- In C#, a top-level *class inside a namespace can be either public or internal. If you do not write anything before class, the default is internal.* Microsoft's C# specification **says** *types declared in namespaces have public or internal access, and the default is internal.*

Example 1: Without internal

```
using System;
```

```
namespace MyApp
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main()
```

```
        {
```

```
            Console.WriteLine("Hello");
```

```
        }
```

```
    }
```

```
}
```

This is automatically treated like:

internal class Program

Example 2: With internal

```
using System;
```

```
namespace MyApp
```

```
{
```

```
    internal class Program
```

```
    {
```

```
        static void Main()
```

```

    {
        Console.WriteLine("Hello");
    }
}

```

Both are okay.

Simple rule

- *Use this for normal beginner programs:*
class Program
- *Use this when you want to clearly say the class is only for the same project:*
internal class Program
- *Use this if another project also needs to use the class:*
public class Program

userdefine input

*In C#, **user-defined input** means the user enters values while the program is running. For console input, use `Console.ReadLine()`, which reads input from the standard input stream.*

Example 1: Input name from user

using System;

```

namespace UserInputExample
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Ask user to enter name
            Console.WriteLine("Enter your name:");

            // Read user input as string
            string name = Console.ReadLine();

            // Display the entered name
            Console.WriteLine("Your name is: " + name);
        }
    }
}

```

Output example

Enter your name:

Ram

Your name is: Ram

Example 2: Input two numbers and add them

Console.ReadLine() gives input as text, so number input must be converted to a numeric type. Microsoft documents `int.Parse()` and `int.TryParse()` as ways to convert a string into an integer. using System;

```
namespace AdditionInputExample
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Ask user for first number
            Console.WriteLine("Enter first number:");

            // Read first input and convert it into integer
            int num1 = int.Parse(Console.ReadLine());

            // Ask user for second number
            Console.WriteLine("Enter second number:");

            // Read second input and convert it into integer
            int num2 = int.Parse(Console.ReadLine());

            // Add two numbers
            int sum = num1 + num2;

            // Display result
            Console.WriteLine("The sum is: " + sum);
        }
    }
}
```

Output example

Enter first number:

10

Enter second number:

20

The sum is: 30

Safer version using TryParse

int.TryParse() is safer because it checks whether conversion is successful instead of directly throwing an error for invalid number input.

using System;

```
namespace SafeInputExample
```

```
{
```

```
    internal class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // Ask user for a number
```

```
            Console.WriteLine("Enter a number:");
```

```
            // Read input as string
```

```
            string input = Console.ReadLine();
```

```
            // Try to convert input into integer
```

```
            bool isNumber = int.TryParse(input, out int number);
```

```
            // Check whether input was valid
```

```
            if (isNumber)
```

```
            {
```

```
                Console.WriteLine("You entered: " + number);
```

```
            }
```

```
            else
```

```
            {
```

```
                Console.WriteLine("Invalid input. Please enter a number.");
```

```
            }
```

```
        }
```

```
    }
```

For beginner programs, you can use int.Parse(). For better programs, use int.TryParse().

For a beginner C# console program, there are mainly 3 common ways:

1. Using Console.ReadLine()

This is the most common way to take user input in a console program. Console.ReadLine() reads one full line of text from the console.

using System;

```
namespace InputExample
```

```

{
    internal class Program
    {
        static void Main()
        {
            Console.WriteLine("Enter your name:");

            string name = Console.ReadLine();

            Console.WriteLine("Your name is: " + name);
        }
    }
}

```

2. Using Console.ReadKey()

Console.ReadKey() reads a single key pressed by the user. It is useful for “Press any key to continue” or menu-type programs. Microsoft describes it as getting the next character or function key pressed by the user.

using System;

namespace KeyInputExample

```

{
    internal class Program
    {
        static void Main()
        {
            Console.WriteLine("Press any key:");

            ConsoleKeyInfo key = Console.ReadKey();

            Console.WriteLine("\nYou pressed: " + key.KeyChar);
        }
    }
}

```

3. Using command-line arguments: string[] args

You can pass input while running the program. Microsoft says string[] args in Main is used to accept command-line arguments.

using System;

namespace ArgumentExample

```

{

```

```

internal class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("First argument is: " + args[0]);
    }
}

```

Run like this:

dotnet run Nepal

Output:

First argument is: Nepal

In C#, input can be understood in two ways:

1. Types of input by method

For beginner console programs, there are mainly **3 input methods**:

Type of input	Method	Use
Line input	Console.ReadLine())	Takes full line input
Key input	Console.ReadKey()	Takes one key press
Command-line input	string[] args	Takes input while running program

Example:

```
// 1. Console.ReadLine()
```

```
string name = Console.ReadLine();
```

```
// 2. Console.ReadKey()
```

```
ConsoleKeyInfo key = Console.ReadKey();
```

```
// 3. Command-line arguments
```

```
static void Main(string[] args)
```

```
{
```

```
    Console.WriteLine(args[0]);
```

```
}
```

2. Types of input by data type

User input can be converted into different data types:

Input type	Example
String	"Ram"
Integer	10
Float/Doubl e	10.5

Character	'A'
Boolean	true or false

Example:

using System;

namespace InputTypesExample

```
{
    internal class Program
    {
        static void Main()
        {
            Console.WriteLine("Enter your name:");
            string name = Console.ReadLine();

            Console.WriteLine("Enter your age:");
            int age = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine("Enter your height:");
            double height = Convert.ToDouble(Console.ReadLine());

            Console.WriteLine("Enter one character:");
            char letter = Convert.ToChar(Console.ReadLine());

            Console.WriteLine("Are you student? true/false:");
            bool isStudent = Convert.ToBoolean(Console.ReadLine());

            Console.WriteLine("Name: " + name);
            Console.WriteLine("Age: " + age);
            Console.WriteLine("Height: " + height);
            Console.WriteLine("Letter: " + letter);
            Console.WriteLine("Student: " + isStudent);
        }
    }
}
```

Simple answer:

Main input methods in console C# = 3

Main input data types = string, int, double, char, bool

3. Basic Language Constructs

Description

These are the foundational syntax elements of C# — variables, data types, operators, and control-flow statements used to write any program logic.

Syntax

// Variable declaration

```
dataType variableName = value;
```

// Conditional

```
if (condition) { } else { }
```

// Loop

```
for (init; condition; update) { }
```

```
while (condition) { }
```

```
do { } while (condition);
```

```
foreach (var item in collection) { }
```

// Switch

```
switch (variable)
```

```
{
```

```
    case value1: break;
```

```
    default: break;
```

```
}
```

Types

Category	Examples
Value Types	int, float, double, char, bool, struct, enum
Reference Types	string, class, array, object, interface
Operators	Arithmetic, Relational, Logical, Assignment, Bitwise
Control Statements	Selection (if, switch), Iteration (for, while, foreach)

Use

- Used to declare data, perform computations, and control the flow of program execution
- Building blocks for every subsequent programming concept

Full Example

```
using System;
```

```
class BasicsDemo
```

```
{
```

```
    static void Main()
```

```
{
```

```

int a = 10, b = 20;
int sum = a + b;

if (sum > 25)
    Console.WriteLine("Sum is greater than 25");
else
    Console.WriteLine("Sum is 25 or less");

for (int i = 1; i <= 5; i++)
{
    Console.WriteLine($"Iteration: {i}");
}

int[] numbers = { 1, 2, 3, 4, 5 };
foreach (int n in numbers)
    Console.WriteLine("Number: " + n);
}
}

```

Typical Assignment Program

Problem: Write a C# program to check whether a number entered by the user is prime or not. using System;

```

class PrimeCheck
{
    static void Main()
    {
        Console.Write("Enter a number: ");
        int num = Convert.ToInt32(Console.ReadLine());
        bool isPrime = true;

        if (num <= 1) isPrime = false;

        for (int i = 2; i <= Math.Sqrt(num); i++)
        {
            if (num % i == 0)
            {
                isPrime = false;
                break;
            }
        }
    }
}

```

```

        Console.WriteLine(isPrime ? $"{num} is Prime" : $"{num} is Not Prime");
    }
}

```

4. Constructor

Description

A constructor is a special method automatically called when an object is instantiated. It has the same name as the class, no return type, and is typically used to initialize object fields.

Syntax

```

class ClassName
{
    public ClassName()           // default constructor
    {
    }

    public ClassName(type param1, ...) // parameterized constructor
    {
    }

    static ClassName()           // static constructor
    {
    }
}

```

Types

Type	Description
Default Constructor	No parameters; auto-generated if none is defined
Parameterized Constructor	Accepts arguments to initialize fields
Copy Constructor	Initializes object using another object's values
Static Constructor	Initializes static members; runs once automatically
Private Constructor	Prevents external instantiation (e.g., Singleton pattern)

Use

- Guarantees an object starts in a valid, initialized state
- Enforces mandatory initialization of data
- Static constructors initialize static/shared data exactly once

Full Example

```

using System;

class Student
{
    public string Name;
}

```

```

public int Age;
public static int TotalStudents;

// Static constructor - runs once
static Student()
{
    TotalStudents = 0;
    Console.WriteLine("Static constructor invoked.");
}

// Default constructor
public Student()
{
    Name = "Unknown";
    Age = 0;
    TotalStudents++;
}

// Parameterized constructor
public Student(string name, int age)
{
    Name = name;
    Age = age;
    TotalStudents++;
}

// Copy constructor
public Student(Student other)
{
    Name = other.Name;
    Age = other.Age;
    TotalStudents++;
}

public void Display() => Console.WriteLine($"Name: {Name}, Age: {Age}");
}

class Program
{
    static void Main()

```

```

{
    Student s1 = new Student("Alice", 20);
    Student s2 = new Student(s1); // copy constructor
    s1.Display();
    s2.Display();
    Console.WriteLine("Total Students: " + Student.TotalStudents);
}
}

```

Typical Assignment Program

Problem: Create a class BankAccount with default, parameterized, and copy constructors. Track the total number of accounts created using a static field.
using System;

```

class BankAccount
{
    public string AccountHolder;
    public double Balance;
    public static int AccountCount = 0;

    public BankAccount()
    {
        AccountHolder = "N/A";
        Balance = 0;
        AccountCount++;
    }

    public BankAccount(string holder, double balance)
    {
        AccountHolder = holder;
        Balance = balance;
        AccountCount++;
    }

    public BankAccount(BankAccount other)
    {
        AccountHolder = other.AccountHolder;
        Balance = other.Balance;
        AccountCount++;
    }
}

```

```

    public void Show() => Console.WriteLine($"{AccountHolder}: Rs. {Balance}");
}

```

```

class Program
{
    static void Main()
    {
        BankAccount a1 = new BankAccount("Ram", 5000);
        BankAccount a2 = new BankAccount(a1);
        a1.Show();
        a2.Show();
        Console.WriteLine("Total Accounts: " + BankAccount.AccountCount);
    }
}

```

5. Properties

Description

Properties expose private fields to the outside world in a controlled way using get and set accessors, supporting encapsulation while looking like public fields to the caller.

Syntax

```
private dataType fieldName;
```

```

public dataType PropertyName
{
    get { return fieldName; }
    set { fieldName = value; }
}

```

// Auto-implemented property

```
public dataType PropertyName { get; set; }
```

Types

Type	Description
Read-Write Property	Has both get and set
Read-Only Property	Only get, no set
Write-Only Property	Only set, no get (rare)
Auto-Implemented Property	Compiler auto-generates hidden backing field
Computed Property	get returns a calculated value, no backing field

Use

- Validate data before storing it (e.g., prevent negative age)
- Hide internal implementation details from external code
- Provide computed/derived values without exposing internal logic

Full Example

using System;

```
class Employee
{
    private double salary;
    private string name;

    public string Name
    {
        get { return name; }
        set { name = value; }
    }

    public double Salary
    {
        get { return salary; }
        set
        {
            if (value < 0)
                Console.WriteLine("Salary cannot be negative.");
            else
                salary = value;
        }
    }

    // Auto-implemented property
    public int Id { get; set; }

    // Computed (read-only) property
    public double AnnualSalary
    {
        get { return salary * 12; }
    }
}
```

```
class Program
```

```

{
    static void Main()
    {
        Employee e = new Employee();
        e.Name = "Sita";
        e.Salary = 30000;
        e.Id = 101;

        Console.WriteLine($"{e.Name} (ID: {e.Id}) earns Rs.{e.Salary}/month,
Rs.{e.AnnualSalary}/year");

        e.Salary = -500; // triggers validation
    }
}

```

Typical Assignment Program

Problem: Create a Rectangle class with Width and Height properties that reject negative values, and a read-only computed property Area.
using System;

```

class Rectangle
{
    private double width, height;

    public double Width
    {
        get { return width; }
        set { width = value < 0 ? 0 : value; }
    }

    public double Height
    {
        get { return height; }
        set { height = value < 0 ? 0 : value; }
    }

    public double Area => width * height; // expression-bodied read-only property
}

class Program
{

```

```

static void Main()
{
    Rectangle r = new Rectangle();
    r.Width = 10;
    r.Height = 5;
    Console.WriteLine("Area: " + r.Area);

    r.Height = -20; // rejected, becomes 0
    Console.WriteLine("Area after invalid height: " + r.Area);
}
}

```

6. Arrays and Strings

Description

Arrays store multiple values of the same type in contiguous memory with a fixed size. **Strings** represent immutable sequences of characters, with rich built-in methods for manipulation.

Syntax

```

// Arrays
dataType[] arrayName = new dataType[size];
dataType[,] array2D = new dataType[rows, cols];
dataType[][] jagged = new dataType[size][];

```

```

// Strings
string s = "text";
StringBuilder sb = new StringBuilder();

```

Types

Type	Description
Single-dimensional array	Simple linear list, e.g., int[] arr
Multi-dimensional array	Grid/matrix, e.g., int[,] arr
Jagged array	Array of arrays, each row can have different length
String (immutable)	Fixed value once created; every modification creates a new object
StringBuilder (mutable)	Efficient for repeated string modifications

Use

- Arrays: store fixed collections of related data (e.g., marks of students)
- Strings: handle text data — names, messages, parsing, formatting
- StringBuilder: efficient string concatenation in loops (avoids excessive memory allocation)

Full Example

```

using System;
using System.Text;

class ArrayStringDemo
{
    static void Main()
    {
        // Single-dimensional array
        int[] marks = { 85, 90, 78, 92, 88 };
        int total = 0;
        foreach (int m in marks) total += m;
        Console.WriteLine("Average marks: " + (total / (double)marks.Length));

        // 2D array
        int[,] matrix = { { 1, 2 }, { 3, 4 } };
        Console.WriteLine("matrix[1,0] = " + matrix[1, 0]);

        // Jagged array
        int[][] jagged = new int[2][];
        jagged[0] = new int[] { 1, 2, 3 };
        jagged[1] = new int[] { 4, 5 };
        Console.WriteLine("jagged[1][1] = " + jagged[1][1]);

        // String operations
        string name = "hamro csit";
        Console.WriteLine(name.ToUpper());
        Console.WriteLine(name.Substring(0, 5));
        Console.WriteLine(name.Replace("hamro", "our"));

        // StringBuilder for efficient concatenation
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i <= 5; i++)
            sb.Append(i).Append(" ");
        Console.WriteLine("Built string: " + sb.ToString());
    }
}

```

Typical Assignment Program

Problem: Write a program to store 5 student names in an array, and marks in another array. Print each student's name with their marks, and find the topper. Also reverse each student's name using string methods.

```

using System;

class StudentMarks
{
    static void Main()
    {
        string[] names = { "Alice", "Bob", "Carol", "David", "Eva" };
        int[] marks = { 78, 92, 85, 92, 67 };

        int topperIndex = 0;
        for (int i = 1; i < marks.Length; i++)
        {
            if (marks[i] > marks[topperIndex])
                topperIndex = i;
        }

        for (int i = 0; i < names.Length; i++)
        {
            char[] chars = names[i].ToCharArray();
            Array.Reverse(chars);
            string reversed = new string(chars);
            Console.WriteLine($"{names[i]} - {marks[i]} marks (reversed: {reversed})");
        }

        Console.WriteLine($"
Topper: {names[topperIndex]} with {marks[topperIndex]} marks");
    }
}

```

7. Indexers

Description

An indexer allows instances of a class to be accessed using array-like syntax (obj[index]), implemented using the this keyword with get/set accessors.

Syntax

```

class ClassName
{
    public dataType this[int index]
    {
        get { /* return value */ }
        set { /* assign value */ }
    }
}

```

```
}
```

Types

Type	Description
Single-parameter indexer	Indexed by one value, e.g., int
Multi-parameter indexer	Indexed by multiple values, e.g., obj[row, col]
Overloaded indexer	Multiple indexers with different parameter types (e.g., int and string)

Use

- Makes custom classes behave like arrays or collections
- Useful for wrapper classes around internal data structures (e.g., a custom dictionary-like class)
- Improves code readability when working with collection-like objects

Full Example

using System;

```
class WeekDays
```

```
{
```

```
    private string[] days = { "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat" };
```

```
    // Integer indexer
```

```
    public string this[int index]
```

```
    {
```

```
        get
```

```
        {
```

```
            if (index >= 0 && index < days.Length)
```

```
                return days[index];
```

```
            return "Invalid Index";
```

```
        }
```

```
        set
```

```
        {
```

```
            if (index >= 0 && index < days.Length)
```

```
                days[index] = value;
```

```
        }
```

```
    }
```

```
    // Overloaded string indexer (search by name, return index)
```

```
    public int this[string dayName]
```

```
    {
```

```
        get { return Array.IndexOf(days, dayName); }
```

```

    }
}

class Program
{
    static void Main()
    {
        WeekDays week = new WeekDays();
        Console.WriteLine(week[0]);    // Sun
        week[0] = "Sunday";
        Console.WriteLine(week[0]);    // Sunday
        Console.WriteLine(week["Tue"]); // index of Tue -> 2
    }
}

```

Typical Assignment Program

Problem: Create a class TemperatureLog that stores temperatures for 7 days of the week and provides an indexer to get/set the temperature by day index (0–6), with bounds checking. using System;

```

class TemperatureLog
{
    private double[] temps = new double[7];

    public double this[int day]
    {
        get
        {
            if (day < 0 || day > 6)
                throw new IndexOutOfRangeException("Day must be between 0 and 6");
            return temps[day];
        }
        set
        {
            if (day < 0 || day > 6)
                throw new IndexOutOfRangeException("Day must be between 0 and 6");
            temps[day] = value;
        }
    }
}

```

```

class Program
{
    static void Main()
    {
        TemperatureLog log = new TemperatureLog();
        log[0] = 25.5;
        log[1] = 27.0;
        Console.WriteLine("Day 0 temp: " + log[0]);
        Console.WriteLine("Day 1 temp: " + log[1]);
    }
}

```

8. Inheritance and the "base" Keyword

Description

Inheritance allows a derived class to acquire the fields, properties, and methods of a base class, enabling code reuse and establishing an "is-a" relationship. The base keyword is used to access base class members from a derived class.

Syntax

```

class BaseClass
{
    public BaseClass() { }
    public void Method() { }
}

class DerivedClass : BaseClass
{
    public DerivedClass() : base() // call base constructor
    { }

    public void CallBaseMethod()
    {
        base.Method(); // call base class method
    }
}

```

Types

Type	Description
Single Inheritance	One class inherits from one base class (only kind C# classes support directly)
Multilevel Inheritance	$A \rightarrow B \rightarrow C$ chain of inheritance
Hierarchical Inheritance	Multiple classes inherit from the same base class

Multiple Inheritance (via interfaces)	A class implements multiple interfaces
---------------------------------------	--

Use

- Reuse common logic across related classes
- Model real-world "is-a" relationships (e.g., Dog is an Animal)
- Reduce code duplication and improve maintainability

Full Example

using System;

```
class Animal
{
    public string Name;

    public Animal(string name)
    {
        Name = name;
        Console.WriteLine("Animal constructor called");
    }

    public void Eat() => Console.WriteLine($"{Name} is eating");
}

class Dog : Animal
{
    public Dog(string name) : base(name) // call base constructor
    {
        Console.WriteLine("Dog constructor called");
    }

    public void Bark()
    {
        base.Eat(); // explicitly call base class method
        Console.WriteLine($"{Name} is barking");
    }
}

class Program
{
    static void Main()
    {

```

```

        Dog d = new Dog("Tommy");
        d.Bark();
    }
}

```

Typical Assignment Program

Problem: Create a base class Vehicle with fields Brand and method Start(). Derive a class Car that adds a NumberOfDoors field and overrides/extends Start() using the base keyword.
using System;

```

class Vehicle
{
    public string Brand;

    public Vehicle(string brand)
    {
        Brand = brand;
    }

    public void Start() => Console.WriteLine($"{Brand} vehicle is starting...");
}

class Car : Vehicle
{
    public int NumberOfDoors;

    public Car(string brand, int doors) : base(brand)
    {
        NumberOfDoors = doors;
    }

    public void StartCar()
    {
        base.Start();
        Console.WriteLine($"Car with {NumberOfDoors} doors is ready to drive.");
    }
}

class Program
{
    static void Main()

```

```

{
    Car myCar = new Car("Toyota", 4);
    myCar.StartCar();
}
}

```

9. Method Hiding and Overriding

Description

Overriding redefines a base class's virtual method in a derived class using override, resolved at runtime (true polymorphism). **Hiding** uses new to hide a base class member with a same-named member in the derived class, resolved at compile time based on the reference type.

Syntax

// Overriding

```

class Base
{
    public virtual void Method() { }
}
class Derived : Base
{
    public override void Method() { }
}

```

// Hiding

```

class Base
{
    public void Method() { }
}
class Derived : Base
{
    public new void Method() { }
}

```

Types

Type	Keyword Used	Resolved At	True Polymorphism?
Overriding	virtual (base) + override (derived)	Runtime	Yes
Hiding	new (derived)	Compile time	No

Use

- Overriding: implement specialized behavior for a base type reference (framework/plugin design)
- Hiding: intentionally provide a new unrelated implementation while keeping the base version accessible via base-type reference

Full Example

using System;

```
class Base
{
    public virtual void Show() => Console.WriteLine("Base Show (virtual)");
    public void Display() => Console.WriteLine("Base Display (non-virtual)");
}

class Derived : Base
{
    public override void Show() => Console.WriteLine("Derived Show (overridden)");
    public new void Display() => Console.WriteLine("Derived Display (hidden)");
}

class Program
{
    static void Main()
    {
        Base b = new Derived();
        b.Show();    // Derived Show (overridden) -- runtime resolution
        b.Display(); // Base Display (non-virtual) -- compile-time resolution (hiding)

        Derived d = new Derived();
        d.Show();    // Derived Show (overridden)
        d.Display(); // Derived Display (hidden)
    }
}
```

Typical Assignment Program

Problem: Create a base class Employee with a virtual method CalculateBonus() and a non-virtual method PrintPolicy(). Derive Manager that overrides CalculateBonus() and hides PrintPolicy() using new. Demonstrate the difference when accessed via a base class reference.

```
using System;

class Employee
{

```

```

    public virtual double CalculateBonus() => 1000;
    public void PrintPolicy() => Console.WriteLine("Standard employee bonus policy");
}

class Manager : Employee
{
    public override double CalculateBonus() => 5000;
    public new void PrintPolicy() => Console.WriteLine("Manager-specific bonus policy");
}

class Program
{
    static void Main()
    {
        Employee e = new Manager();
        Console.WriteLine("Bonus: " + e.CalculateBonus()); // 5000 (overridden - runtime)
        e.PrintPolicy(); // Standard... (hidden - compile-time)

        Manager m = new Manager();
        m.PrintPolicy(); // Manager-specific...
    }
}

```

10. Applying Polymorphism in Code Extensibility

Description

Polymorphism allows objects of different derived types to be treated uniformly through a common base type/interface reference, while each executes its own specific behavior — enabling systems to be extended with new types without modifying existing code.

Syntax

```

BaseType obj = new DerivedType();
obj.Method(); // calls the derived type's overridden implementation

```

Types

Type	Description
Compile-time (Static) Polymorphism	Method overloading, operator overloading — resolved at compile time
Runtime (Dynamic) Polymorphism	Method overriding via virtual/override, interfaces — resolved at runtime

Use

- Add new derived classes (e.g., new shape types) without touching existing code that uses the base type — supports the **Open/Closed Principle**

- Build extensible frameworks, plugin architectures, and flexible APIs
- Simplify code that processes collections of mixed but related object types

Full Example

```
using System;
```

```
using System.Collections.Generic;
```

```
abstract class Shape
```

```
{
    public abstract double Area();
}
```

```
class Circle : Shape
```

```
{
    public double Radius;
    public Circle(double r) { Radius = r; }
    public override double Area() => Math.PI * Radius * Radius;
}
```

```
class Rectangle : Shape
```

```
{
    public double Width, Height;
    public Rectangle(double w, double h) { Width = w; Height = h; }
    public override double Area() => Width * Height;
}
```

```
// New shape added later - WITHOUT modifying existing code
```

```
class Triangle : Shape
```

```
{
    public double Base, Height;
    public Triangle(double b, double h) { Base = b; Height = h; }
    public override double Area() => 0.5 * Base * Height;
}
```

```
class Program
```

```
{
    static void Main()
    {
        List<Shape> shapes = new List<Shape>
        {
            new Circle(5),

```

```

        new Rectangle(4, 6),
        new Triangle(3, 8)
    };

    foreach (Shape s in shapes)
        Console.WriteLine($"{s.GetType().Name} Area: {s.Area():F2}");
    }
}

```

Typical Assignment Program

Problem: Design a PaymentMethod abstract class with method Pay(double amount). Create CreditCardPayment, CashPayment, and MobileWalletPayment classes that each implement Pay() differently. Process a list of mixed payments polymorphically.

```
using System;
```

```
using System.Collections.Generic;
```

```

abstract class PaymentMethod
{
    public abstract void Pay(double amount);
}

class CreditCardPayment : PaymentMethod
{
    public override void Pay(double amount) =>
        Console.WriteLine($"Paid Rs. {amount} using Credit Card");
}

class CashPayment : PaymentMethod
{
    public override void Pay(double amount) =>
        Console.WriteLine($"Paid Rs. {amount} in Cash");
}

class MobileWalletPayment : PaymentMethod
{
    public override void Pay(double amount) =>
        Console.WriteLine($"Paid Rs. {amount} via Mobile Wallet");
}

class Program
{

```

```

static void Main()
{
    List<PaymentMethod> payments = new List<PaymentMethod>
    {
        new CreditCardPayment(),
        new CashPayment(),
        new MobileWalletPayment()
    };

    foreach (var p in payments)
        p.Pay(1500);
}
}

```

11. Structs and Enums

Description

A **struct** is a lightweight value type ideal for small data groupings (e.g., a point or a date). An **enum** defines a set of named integral constants to make code more readable and self-documenting.

Syntax

```

// Struct
struct StructName
{
    public dataType field;
    public StructName(dataType value) { field = value; }
}

```

```

// Enum
enum EnumName { Value1, Value2, Value3 }

```

Types

Type	Description
Struct	Value type; stack-allocated; no inheritance from classes; can implement interfaces
Enum	Named integer constants; default underlying type int (can be byte, long, etc.)
Flags Enum	Enum marked with [Flags] attribute to allow bitwise combination of values

Use

- Structs: represent small, immutable-like data (coordinates, RGB colors, date ranges) with value semantics

- Enums: replace "magic numbers" with meaningful names (e.g., Days.Monday instead of 1)

Full Example

using System;

```
struct Point
```

```
{
    public int X, Y;
    public Point(int x, int y) { X = x; Y = y; }
    public void Display() => Console.WriteLine($"({X}, {Y})");
}
```

```
enum Days { Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday }
```

[Flags]

```
enum Permissions
```

```
{
    None = 0,
    Read = 1,
    Write = 2,
    Execute = 4
}
```

```
class Program
```

```
{
    static void Main()
    {
        Point p1 = new Point(3, 4);
        p1.Display();

        Point p2 = p1; // copied by VALUE (struct)
        p2.X = 100;
        p1.Display(); // unaffected: (3, 4)
        p2.Display(); // (100, 4)

        Days today = Days.Wednesday;
        Console.WriteLine($"Today is {today}, index {(int)today}");

        Permissions userPerm = Permissions.Read | Permissions.Write;
        Console.WriteLine("Permissions: " + userPerm);
    }
}
```

```
}  
}
```

Typical Assignment Program

Problem: Create a struct Complex to represent a complex number (real, imaginary) with a method to add two complex numbers. Create an enum Operation (Add, Subtract, Multiply, Divide) and use it in a simple calculator.

using System;

```
struct Complex
```

```
{  
    public double Real, Imaginary;  
    public Complex(double r, double i) { Real = r; Imaginary = i; }  
  
    public static Complex Add(Complex a, Complex b) =>  
        new Complex(a.Real + b.Real, a.Imaginary + b.Imaginary);  
  
    public void Display() => Console.WriteLine($"{Real} + {Imaginary}i");  
}
```

```
enum Operation { Add, Subtract, Multiply, Divide }
```

```
class Program
```

```
{  
    static double Calculate(double a, double b, Operation op)  
    {  
        switch (op)  
        {  
            case Operation.Add: return a + b;  
            case Operation.Subtract: return a - b;  
            case Operation.Multiply: return a * b;  
            case Operation.Divide: return b != 0 ? a / b : double.NaN;  
            default: return 0;  
        }  
    }  
}
```

```
static void Main()
```

```
{  
    Complex c1 = new Complex(2, 3);  
    Complex c2 = new Complex(1, 4);  
    Complex sum = Complex.Add(c1, c2);  
}
```

```

        sum.Display(); // 3 + 7i

        Console.WriteLine("10 / 2 = " + Calculate(10, 2, Operation.Divide));
    }
}

```

12. Abstract Class and Sealed Class

Description

An **abstract class** cannot be instantiated and may contain both abstract (unimplemented) and concrete methods; it serves as a base for derived classes. A **sealed class** cannot be inherited at all — used to prevent further extension.

Syntax

```

abstract class AbstractClass
{
    public abstract void AbstractMethod(); // no body
    public void ConcreteMethod() { }      // has body
}

```

```

sealed class SealedClass
{
    public void Method() { }
}
// class X : SealedClass { } // COMPILE ERROR

```

Types

Type	Description
Abstract Class	Partial implementation; forces derived classes to implement abstract members
Sealed Class	Complete/final implementation; cannot be extended
Sealed Method	A method overridden in a derived class marked sealed override cannot be overridden further down the chain

Use

- Abstract class: define a common template/contract while sharing some default behavior (e.g., Shape base with common Display(), abstract Area())
- Sealed class: protect critical/security-sensitive classes from being modified via inheritance, or optimize performance (JIT can make some optimizations for sealed classes)

Full Example

```
using System;
```

```
abstract class Shape
```

```

{
    public abstract double Area();      // must be implemented
    public void Display() => Console.WriteLine($"Area = {Area():F2}"); // concrete
}

class Circle : Shape
{
    public double Radius;
    public Circle(double r) { Radius = r; }
    public override double Area() => Math.PI * Radius * Radius;
}

sealed class FinalCircle : Circle
{
    public FinalCircle(double r) : base(r) { }
    // No further derived class can inherit FinalCircle
}

// class ExtendedCircle : FinalCircle { } // COMPILE ERROR: cannot inherit sealed class

class Program
{
    static void Main()
    {
        Shape s = new Circle(5);
        s.Display();

        FinalCircle fc = new FinalCircle(3);
        fc.Display();
    }
}

```

Typical Assignment Program

Problem: Create an abstract class Employee with abstract method CalculateSalary() and concrete method DisplayDetails(). Derive PermanentEmployee and ContractEmployee. Make ContractEmployee a sealed class so it cannot be further extended.
using System;

```

abstract class Employee
{
    public string Name;

```

```

public Employee(string name) { Name = name; }

public abstract double CalculateSalary();

public void DisplayDetails() =>
    Console.WriteLine($"{Name}: Salary = Rs. {CalculateSalary()}");
}

class PermanentEmployee : Employee
{
    public double BaseSalary;
    public PermanentEmployee(string name, double baseSalary) : base(name)
    {
        BaseSalary = baseSalary;
    }
    public override double CalculateSalary() => BaseSalary + 5000; // fixed bonus
}

sealed class ContractEmployee : Employee
{
    public double HourlyRate;
    public int HoursWorked;

    public ContractEmployee(string name, double rate, int hours) : base(name)
    {
        HourlyRate = rate;
        HoursWorked = hours;
    }
    public override double CalculateSalary() => HourlyRate * HoursWorked;
}

class Program
{
    static void Main()
    {
        Employee e1 = new PermanentEmployee("Ram", 20000);
        Employee e2 = new ContractEmployee("Shyam", 500, 80);

        e1.DisplayDetails();
        e2.DisplayDetails();
    }
}

```

```
}  
}
```

13. Interface

Description

An interface defines a contract of members (methods, properties, events, indexers) that implementing classes/structs must provide. It supports full abstraction and allows a class to "implement" multiple interfaces (a workaround for C#'s single-class-inheritance restriction).

Syntax

```
interface IInterfaceName
```

```
{  
    void Method1();  
    dataType Property1 { get; set; }  
}
```

```
class ClassName : IInterfaceName
```

```
{  
    public void Method1() { }  
    public dataType Property1 { get; set; }  
}
```

Types

Type	Description
Standard Interface	Only member signatures, no implementation (pre-C# 8)
Interface with Default Implementation	(C# 8+) can provide a default method body
Multiple Interface Implementation	A class implements more than one interface

Use

- Achieve full abstraction and loose coupling between components
- Allow unrelated classes to share common behavior/contract (e.g., IComparable, IDisposable)
- Enable multiple inheritance-like behavior in C#

Full Example

```
using System;
```

```
interface IShape
```

```
{  
    double Area();  
    double Perimeter();  
}
```

```

interface IPrintable
{
    void Print();
}

// Implements multiple interfaces
class Rectangle : IShape, IPrintable
{
    public double Width, Height;
    public Rectangle(double w, double h) { Width = w; Height = h; }

    public double Area() => Width * Height;
    public double Perimeter() => 2 * (Width + Height);
    public void Print() => Console.WriteLine($"Rectangle {Width}x{Height}");
}

class Program
{
    static void Main()
    {
        Rectangle r = new Rectangle(5, 3);
        r.Print();
        Console.WriteLine("Area: " + r.Area());
        Console.WriteLine("Perimeter: " + r.Perimeter());

        IShape shapeRef = r; // interface reference
        Console.WriteLine("Via interface -> Area: " + shapeRef.Area());
    }
}

```

Typical Assignment Program

Problem: Define interfaces IMovable (with Move()) and IFlyable (with Fly()). Create a class Bird that implements both, and a class Car that implements only IMovable. Demonstrate polymorphic use via interface references.

using System;

```

interface IMovable
{
    void Move();
}

```

```

interface IFlyable
{
    void Fly();
}

class Bird : IMovable, IFlyable
{
    public void Move() => Console.WriteLine("Bird is walking");
    public void Fly() => Console.WriteLine("Bird is flying");
}

class Car : IMovable
{
    public void Move() => Console.WriteLine("Car is driving");
}

class Program
{
    static void Main()
    {
        IMovable[] movables = { new Bird(), new Car() };
        foreach (var m in movables)
            m.Move();

        IFlyable flyer = new Bird();
        flyer.Fly();
    }
}

```

14. Delegate and Events

Description

A **delegate** is a type-safe object that holds a reference to a method, allowing methods to be passed as parameters and invoked indirectly. An **event** is built on top of a delegate and implements the publisher-subscriber (observer) pattern, restricting external code to only subscribe/unsubscribe (not invoke or overwrite directly).

Syntax

```

// Delegate
delegate returnType DelegateName(parameters);
DelegateName d = MethodName;

```

```
d(args); // invoke
```

```
// Event
class Publisher
{
    public event DelegateName EventName;
    void RaiseEvent() { EventName?.Invoke(args); }
}
```

Types

Type	Description
Single-cast Delegate	Refers to a single method
Multicast Delegate	Refers to multiple methods via +=, invoked in sequence
Built-in Delegates	Func<T,TResult>, Action<T>, Predicate<T>
Event	Special delegate wrapper restricting external invocation

Use

- Pass methods as arguments (callback mechanisms)
- Implement event-driven programming (e.g., button click handlers in GUI apps)
- Decouple the component raising an event from the components handling it

Full Example

```
using System;
```

```
// Custom delegate
delegate void NotifyHandler(string message);

class Publisher
{
    // Event based on the delegate
    public event NotifyHandler OnStatusChanged;

    public void ChangeStatus(string status)
    {
        Console.WriteLine("Status changing...");
        OnStatusChanged?.Invoke(status); // raise event
    }
}

class Subscriber
{
```

```

    public void HandleStatusChange(string message)
    {
        Console.WriteLine("Subscriber received: " + message);
    }
}

class Program
{
    static void Main()
    {
        Publisher pub = new Publisher();
        Subscriber sub = new Subscriber();

        pub.OnStatusChanged += sub.HandleStatusChange; // subscribe
        pub.OnStatusChanged += msg => Console.WriteLine("Lambda subscriber: " + msg); //
multicast

        pub.ChangeStatus("Order Shipped");

        // Built-in delegate examples
        Func<int, int, int> multiply = (a, b) => a * b;
        Console.WriteLine("5 x 3 = " + multiply(5, 3));

        Action<string> log = msg => Console.WriteLine("LOG: " + msg);
        log("Application started");
    }
}

```

Typical Assignment Program

Problem: Simulate a simple alarm system: create an Alarm class that raises an event OnAlarmTriggered when Trigger() is called. Create two subscribers: one that logs the alarm, and one that sends a (simulated) notification.
using System;

```

delegate void AlarmHandler(string location);

class Alarm
{
    public event AlarmHandler OnAlarmTriggered;

    public void Trigger(string location)

```

```

    {
        Console.WriteLine("Alarm triggered!");
        OnAlarmTriggered?.Invoke(location);
    }
}

class Logger
{
    public void LogAlarm(string location) =>
        Console.WriteLine($"[LOG] Alarm at {location} recorded.");
}

class NotificationService
{
    public void SendNotification(string location) =>
        Console.WriteLine($"[NOTIFY] Sending SMS: Intrusion detected at {location}!");
}

class Program
{
    static void Main()
    {
        Alarm alarm = new Alarm();
        Logger logger = new Logger();
        NotificationService notifier = new NotificationService();

        alarm.OnAlarmTriggered += logger.LogAlarm;
        alarm.OnAlarmTriggered += notifier.SendNotification;

        alarm.Trigger("Main Gate");
    }
}

```

15. Partial Class

Description

A partial class lets the definition of a single class be split across multiple physical files (each marked with partial); the compiler merges them into one class at compile time.

Syntax

```

// File1.cs
public partial class ClassName

```

```
{
    // part 1 members
}
```

```
// File2.cs
public partial class ClassName
{
    // part 2 members
}
```

Types

Type	Description
Partial Class	Split class definition
Partial Method	A method declared in one part, optionally implemented in another; if unimplemented, it's removed at compile time
Partial Struct/Interface	Same concept applied to structs and interfaces

Use

- Separate auto-generated code (e.g., Designer files, Entity Framework scaffolding) from hand-written code
- Allow multiple developers to work on different parts of a large class simultaneously
- Improve organization of large classes by logical grouping

Full Example

```
// PersonPart1.cs
using System;
```

```
public partial class Person
{
    public string Name;
    public int Age;

    public void DisplayBasicInfo()
    {
        Console.WriteLine($"Name: {Name}, Age: {Age}");
    }
}
```

```
// PersonPart2.cs
public partial class Person
{
    public string Address;
```

```

    public void DisplayFullInfo()
    {
        DisplayBasicInfo();
        Console.WriteLine($"Address: {Address}");
    }
}

class Program
{
    static void Main()
    {
        Person p = new Person { Name = "Hari", Age = 22, Address = "Kathmandu" };
        p.DisplayFullInfo();
    }
}

```

Typical Assignment Program

Problem: Split a Car class into two partial parts — one containing basic details (Model, Year) in one section, and another containing engine-related details (EngineType, HorsePower) with a method that displays all information combined.

using System;

// Part 1

```

public partial class Car
{
    public string Model;
    public int Year;
}

```

// Part 2

```

public partial class Car
{
    public string EngineType;
    public int HorsePower;

    public void DisplayFullSpec()
    {
        Console.WriteLine($"{Year} {Model} - {EngineType} Engine, {HorsePower} HP");
    }
}

```

```

class Program
{
    static void Main()
    {
        Car car = new Car
        {
            Model = "Civic",
            Year = 2023,
            EngineType = "Hybrid",
            HorsePower = 180
        };
        car.DisplayFullSpec();
    }
}

```

16. Collections

Description

Collections are dynamic, resizable groupings of objects — more flexible than fixed-size arrays. C# provides both non-generic (System.Collections) and generic (System.Collections.Generic) collection classes.

Syntax

```

List<T> list = new List<T>();
Dictionary<TKey, TValue> dict = new Dictionary<TKey, TValue>();
Queue<T> queue = new Queue<T>();
Stack<T> stack = new Stack<T>();
HashSet<T> set = new HashSet<T>();

```

Types

Type	Description
List<T>	Dynamic array; ordered, allows duplicates
Dictionary<TKey,TValue>	Key-value pairs; fast lookup by key
Queue<T>	FIFO (First-In-First-Out) collection
Stack<T>	LIFO (Last-In-First-Out) collection
HashSet<T>	Unordered, no duplicate elements
ArrayList, Hashtable (non-generic) Store object type; not type-safe (legacy)	

Use

- List<T>: dynamic lists of items (e.g., student records)
- Dictionary<TKey,TValue>: fast key-based lookups (e.g., username → user object)
- Queue<T>: task scheduling, print queues, breadth-first processing

- Stack<T>: undo operations, expression evaluation, depth-first processing

Full Example

using System;

using System.Collections.Generic;

```
class CollectionsDemo
{
    static void Main()
    {
        // List
        List<string> fruits = new List<string> { "Apple", "Banana" };
        fruits.Add("Mango");
        fruits.Remove("Banana");
        Console.WriteLine("Fruits: " + string.Join(", ", fruits));

        // Dictionary
        Dictionary<string, int> ages = new Dictionary<string, int>();
        ages["Ram"] = 25;
        ages["Sita"] = 22;
        foreach (var kv in ages)
            Console.WriteLine($"{kv.Key} is {kv.Value} years old");

        // Queue
        Queue<string> printQueue = new Queue<string>();
        printQueue.Enqueue("Doc1");
        printQueue.Enqueue("Doc2");
        Console.WriteLine("Printing: " + printQueue.Dequeue());

        // Stack
        Stack<int> undoStack = new Stack<int>();
        undoStack.Push(1);
        undoStack.Push(2);
        Console.WriteLine("Undo: " + undoStack.Pop());

        // HashSet
        HashSet<int> uniqueNumbers = new HashSet<int> { 1, 2, 2, 3 };
        Console.WriteLine("Unique count: " + uniqueNumbers.Count);
    }
}
```

Typical Assignment Program

Problem: Build a simple contact book using a Dictionary<string,string> (name → phone number). Allow adding, searching, and removing a contact. Also implement a Queue<string> to simulate a customer service ticket system (add/serve tickets).

using System;

using System.Collections.Generic;

class Program

```
{
    static void Main()
    {
        // Contact Book
        Dictionary<string, string> contacts = new Dictionary<string, string>();
        contacts["Ram"] = "9800000001";
        contacts["Sita"] = "9800000002";
        contacts.Remove("Sita");

        if (contacts.ContainsKey("Ram"))
            Console.WriteLine("Ram's number: " + contacts["Ram"]);

        // Ticket Queue System
        Queue<string> tickets = new Queue<string>();
        tickets.Enqueue("Ticket#101");
        tickets.Enqueue("Ticket#102");
        tickets.Enqueue("Ticket#103");

        Console.WriteLine("Serving: " + tickets.Dequeue());
        Console.WriteLine("Next in queue: " + tickets.Peek());
        Console.WriteLine("Remaining tickets: " + tickets.Count);
    }
}
```

17. Generics

Description

Generics allow classes, methods, and interfaces to be defined with a placeholder **type parameter** (e.g., T), which is specified when the generic type is used, providing compile-time type safety and reusability without duplicating code for each data type.

Syntax

```
// Generic class
class ClassName<T>
{
```

```

    public T Field;
    public void Method(T param) { }
}

// Generic method
static void MethodName<T>(T param) { }

```

```

// Constrained generic
class ClassName<T> where T : BaseClass, new()

```

Types

Type	Description
Generic Class	e.g., List<T>, custom Box<T>
Generic Method	Method with its own type parameter, independent of enclosing class
Generic Interface	e.g., IComparable<T>, IEnumerable<T>
Constrained Generics	Restrict T using where T : class/struct/new()/BaseType

Use

- Write one reusable class/method that works with any data type safely (e.g., a generic Stack<T>)
- Avoid boxing/unboxing overhead compared to using object types
- Catch type mismatch errors at compile time instead of runtime

Full Example

using System;

```

class GenericBox<T>
{
    private T item;

    public void Add(T value) => item = value;
    public T Get() => item;
    public void Display() => Console.WriteLine($"Stored value: {item}");
}

```

```

// Generic method
class Utility
{
    public static void Swap<T>(ref T a, ref T b)
    {
        T temp = a;

```

```

        a = b;
        b = temp;
    }
}

```

```

// Constrained generic
class Repository<T> where T : new()
{
    public T CreateInstance() => new T();
}

```

```

class Program
{
    static void Main()
    {
        GenericBox<int> intBox = new GenericBox<int>();
        intBox.Add(100);
        intBox.Display();

        GenericBox<string> strBox = new GenericBox<string>();
        strBox.Add("Hello Generics");
        strBox.Display();

        int x = 5, y = 10;
        Utility.Swap(ref x, ref y);
        Console.WriteLine($"x={x}, y={y}");
    }
}

```

Typical Assignment Program

Problem: Create a generic class Pair<T1, T2> that stores two related values of possibly different types, with a method to display them. Also write a generic method FindMax<T> that returns the maximum of two comparable values.

```

using System;
using System.Collections.Generic;

```

```

class Pair<T1, T2>
{
    public T1 First;
    public T2 Second;
}

```

```

    public Pair(T1 first, T2 second)
    {
        First = first;
        Second = second;
    }

    public void Display() => Console.WriteLine($"({First}, {Second})");
}

class Utility
{
    public static T FindMax<T>(T a, T b) where T : IComparable<T>
    {
        return a.CompareTo(b) > 0 ? a : b;
    }
}

class Program
{
    static void Main()
    {
        Pair<string, int> studentAge = new Pair<string, int>("Alice", 21);
        studentAge.Display();

        Console.WriteLine("Max: " + Utility.FindMax(15, 42));
        Console.WriteLine("Max: " + Utility.FindMax("apple", "banana"));
    }
}

```

18. File I/O

Description

File I/O (Input/Output) operations, provided via the System.IO namespace, allow programs to create, read, write, and manipulate files and directories on disk.

Syntax

```

// Simple whole-file operations
File.WriteAllText(path, content);
string content = File.ReadAllText(path);
File.AppendAllText(path, moreContent);

// Stream-based (recommended for large files/line-by-line)

```

```

using (StreamWriter sw = new StreamWriter(path))
{
    sw.WriteLine(text);
}
using (StreamReader sr = new StreamReader(path))
{
    string line = sr.ReadLine();
}

```

Types

Class	Purpose
File / FileInfo	Static/instance methods for file operations (create, copy, delete, move)
Directory / DirectoryInfo	Operations on folders
StreamReader / StreamWriter	Text-based reading/writing
FileStream	Low-level byte-based reading/writing
BinaryReader / BinaryWriter	Read/write binary data

Use

- Persist application data (logs, configuration, reports) between program runs
- Read input data from external text/CSV files for processing
- Store and retrieve structured or unstructured content on disk

Full Example

```

using System;
using System.IO;

class FileIODemo
{
    static void Main()
    {
        string path = "students.txt";

        // Writing using StreamWriter
        using (StreamWriter sw = new StreamWriter(path))
        {
            sw.WriteLine("Alice,85");
            sw.WriteLine("Bob,90");
            sw.WriteLine("Carol,78");
        }
    }
}

```

```

// Reading using StreamReader
Console.WriteLine("File contents:");
using (StreamReader sr = new StreamReader(path))
{
    string line;
    while ((line = sr.ReadLine()) != null)
    {
        string[] parts = line.Split(',');
        Console.WriteLine($"Name: {parts[0]}, Marks: {parts[1]}");
    }
}

// Appending
File.AppendAllText(path, "David,88\n");

// Check existence & delete demo
if (File.Exists(path))
    Console.WriteLine("File exists at: " + Path.GetFullPath(path));
}
}

```

Typical Assignment Program

Problem: Write a program that takes student names and marks from the user, writes them to a file records.txt, then reads the file back, calculates and displays the average mark.

```

using System;
using System.IO;

class Program
{
    static void Main()
    {
        string path = "records.txt";

        using (StreamWriter sw = new StreamWriter(path))
        {
            Console.Write("How many students? ");
            int n = Convert.ToInt32(Console.ReadLine());

            for (int i = 0; i < n; i++)
            {
                Console.Write($"Enter name and marks for student {i + 1} (comma separated): ");
            }
        }
    }
}

```

```

        string entry = Console.ReadLine();
        sw.WriteLine(entry);
    }
}

// Read back and calculate average
int total = 0, count = 0;
using (StreamReader sr = new StreamReader(path))
{
    string line;
    while ((line = sr.ReadLine()) != null)
    {
        string[] parts = line.Split(',');
        Console.WriteLine($"Name: {parts[0].Trim()}, Marks: {parts[1].Trim()}");
        total += Convert.ToInt32(parts[1].Trim());
        count++;
    }
}
Console.WriteLine($"Average Marks: {(double)total / count:F2}");
}
}

```

19. LINQ Fundamentals: Lambda Expressions

Description

LINQ (Language Integrated Query) provides SQL-like query capabilities directly in C#, working over collections, arrays, XML, and databases. **Lambda expressions** ($\Rightarrow$) are anonymous inline functions heavily used with LINQ and delegates.

Syntax

```

// Lambda expression
(parameters) => expression_or_block

```

```

// LINQ Query syntax
var result = from item in collection
              where condition
              select item;

```

```

// LINQ Method syntax
var result = collection.Where(item => condition).Select(item => item);

```

Types

Type	Description
------	-------------

Query Syntax	SQL-like: from...where...select
Method Syntax	Fluent/chained: .Where().Select().OrderBy()
Lambda (Expression body)	<code>x => x * 2</code>
Lambda (Statement body)	<code>x => { statements; return value; }</code>
Common LINQ operators	Where, Select, OrderBy, GroupBy, Sum, Average, Count, First, Any, All

Use

- Filter, sort, transform, and aggregate collections concisely
- Replace verbose loops with declarative, readable queries
- Query databases (via Entity Framework), XML, and in-memory objects with unified syntax

Full Example

```
using System;
using System.Linq;
using System.Collections.Generic;
```

```
class LinqDemo
{
    static void Main()
    {
        List<int> numbers = new List<int> { 5, 12, 8, 21, 3, 17, 9 };

        // Lambda expressions with built-in delegates
        Func<int, bool> isEven = n => n % 2 == 0;
        Console.WriteLine("Is 8 even? " + isEven(8));

        // LINQ Method syntax
        var evens = numbers.Where(n => n % 2 == 0).OrderBy(n => n);
        Console.WriteLine("Even numbers (sorted): " + string.Join(", ", evens));

        // LINQ Query syntax
        var greaterThanTen = from n in numbers
                             where n > 10
                             orderby n descending
                             select n;
        Console.WriteLine("Numbers > 10: " + string.Join(", ", greaterThanTen));

        // Aggregation
```

```

Console.WriteLine("Sum: " + numbers.Sum());
Console.WriteLine("Average: " + numbers.Average());
Console.WriteLine("Max: " + numbers.Max());

// Select (projection)
var squares = numbers.Select(n => n * n);
Console.WriteLine("Squares: " + string.Join(", ", squares));
}
}

```

Typical Assignment Program

Problem: Given a list of Student objects (Name, Marks), use LINQ to: (a) find all students who scored above 75, (b) sort them by marks descending, (c) calculate the class average, (d) find the topper's name using lambda expressions.

```

using System;
using System.Linq;
using System.Collections.Generic;

class Student
{
    public string Name;
    public int Marks;
    public Student(string name, int marks) { Name = name; Marks = marks; }
}

class Program
{
    static void Main()
    {
        List<Student> students = new List<Student>
        {
            new Student("Alice", 82),
            new Student("Bob", 68),
            new Student("Carol", 91),
            new Student("David", 74),
            new Student("Eva", 88)
        };

        // (a) Above 75, (b) sorted descending
        var topStudents = students.Where(s => s.Marks > 75)
            .OrderByDescending(s => s.Marks);
    }
}

```

```

Console.WriteLine("Students scoring above 75:");
foreach (var s in topStudents)
    Console.WriteLine($"{s.Name}: {s.Marks}");

// (c) Class average
double average = students.Average(s => s.Marks);
Console.WriteLine($"Class Average: {average:F2}");

// (d) Topper
Student topper = students.OrderByDescending(s => s.Marks).First();
Console.WriteLine($"Topper: {topper.Name} with {topper.Marks} marks");
}
}

```

20. Try Statements and Exceptions

Description

Exception handling manages runtime errors gracefully using try (code that might fail), catch (handles specific exception types), finally (always executes, e.g., for cleanup), and throw (raises an exception, including custom ones).

Syntax

```

try
{
    // risky code
}
catch (SpecificException ex)
{
    // handle specific exception
}
catch (Exception ex)
{
    // handle general exception
}
finally
{
    // always executes
}

```

```
throw new ExceptionType("message");
```

Types

Type	Description
Built-in Exceptions	DivideByZeroException, NullReferenceException, IndexOutOfRangeException, FormatException, IOException
Custom Exceptions	User-defined classes derived from Exception
Nested try-catch	A try block inside another try/catch
throw vs throw ex	throw; preserves original stack trace; throw ex; resets it

Use

- Prevent program crashes due to unexpected runtime errors
- Provide meaningful error messages/recovery instead of abrupt termination
- Guarantee cleanup code (closing files/connections) always runs via finally

Full Example

using System;

```

class InsufficientBalanceException : Exception
{
    public InsufficientBalanceException(string message) : base(message) { }
}

class BankAccount
{
    public double Balance = 1000;

    public void Withdraw(double amount)
    {
        if (amount > Balance)
            throw new InsufficientBalanceException("Insufficient balance for withdrawal.");
        Balance -= amount;
        Console.WriteLine($"Withdrew {amount}. Remaining balance: {Balance}");
    }
}

class Program
{
    static void Main()
    {
        BankAccount acc = new BankAccount();
    }
}

```

```

try
{
    Console.WriteLine("Enter withdrawal amount: ");
    double amount = Convert.ToDouble(Console.ReadLine());
    acc.Withdraw(amount);

    int[] arr = new int[3];
    Console.WriteLine(arr[5]); // triggers IndexOutOfRangeException
}
catch (InsufficientBalanceException ex)
{
    Console.WriteLine("Custom Error: " + ex.Message);
}
catch (FormatException)
{
    Console.WriteLine("Please enter a valid number.");
}
catch (IndexOutOfRangeException ex)
{
    Console.WriteLine("Array error: " + ex.Message);
}
catch (Exception ex)
{
    Console.WriteLine("Unexpected error: " + ex.Message);
}
finally
{
    Console.WriteLine("Transaction attempt completed.");
}
}
}

```

Typical Assignment Program

Problem: Write a program that reads two integers and an operator (+, -, *, /) from the user and performs the operation, handling DivideByZeroException, FormatException, and a custom InvalidOperatorException for unsupported operators.

using System;

```

class InvalidOperatorException : Exception
{
    public InvalidOperatorException(string message) : base(message) { }
}

```

```
}
```

```
class Program
```

```
{
```

```
    static double Calculate(double a, double b, string op)
```

```
    {
```

```
        switch (op)
```

```
        {
```

```
            case "+": return a + b;
```

```
            case "-": return a - b;
```

```
            case "*": return a * b;
```

```
            case "/":
```

```
                if (b == 0) throw new DivideByZeroException("Cannot divide by zero.");
```

```
                return a / b;
```

```
            default:
```

```
                throw new InvalidOperatorException($"Operator '{op}' is not supported.");
```

```
        }
```

```
    }
```

```
static void Main()
```

```
{
```

```
    try
```

```
    {
```

```
        Console.Write("Enter first number: ");
```

```
        double a = Convert.ToDouble(Console.ReadLine());
```

```
        Console.Write("Enter operator (+,-,*,/): ");
```

```
        string op = Console.ReadLine();
```

```
        Console.Write("Enter second number: ");
```

```
        double b = Convert.ToDouble(Console.ReadLine());
```

```
        double result = Calculate(a, b, op);
```

```
        Console.WriteLine("Result: " + result);
```

```
    }
```

```
    catch (FormatException)
```

```
    {
```

```
        Console.WriteLine("Error: Please enter valid numbers.");
```

```
    }
```

```
    catch (DivideByZeroException ex)
```

```
    {
```

```
        Console.WriteLine("Error: " + ex.Message);
```

```

    }
    catch (InvalidOperationException ex)
    {
        Console.WriteLine("Error: " + ex.Message);
    }
    finally
    {
        Console.WriteLine("Calculation attempt finished.");
    }
}
}

```

21. Attributes

Description

Attributes attach declarative metadata to code elements (classes, methods, properties, assemblies), which can be inspected at runtime via **Reflection**. They are themselves classes derived from `System.Attribute`.

Syntax

```

[AttributeName]
[AttributeName(positionalParam, NamedParam = value)]
public class/method/property Target { }

```

```

// Custom attribute class
[AttributeUsage(AttributeTargets.Method)]
class CustomAttribute : Attribute
{
    public CustomAttribute(string param) { }
    public string NamedParam { get; set; }
}

```

Types

Type	Description
Predefined Attributes	[Obsolete], [Serializable], [Conditional], [DllImport]
Custom Attributes	User-defined, derived from <code>System.Attribute</code>
Positional Parameters	Passed via constructor, e.g., [Author("John")]
Named Parameters	Set via property syntax, e.g., [Author(Version = 2)]
Attribute Targets	Restrict where an attribute applies: Class, Method, Property, Field, All, etc.

Use

- Mark deprecated code ([Obsolete]) to warn developers
- Enable serialization ([Serializable])
- Build custom metadata for frameworks (e.g., validation attributes, ORM mapping, unit test discovery)
- Read via Reflection at runtime to drive dynamic behavior

Full Example

```
using System;
```

```
using System.Reflection;
```

```
[AttributeUsage(AttributeTargets.Method)]
```

```
class AuthorAttribute : Attribute
```

```
{
```

```
    public string Name;
```

```
    public int Version { get; set; }
```

```
    public AuthorAttribute(string name) // positional parameter
```

```
    {
```

```
        Name = name;
```

```
    }
```

```
}
```

```
class Program
```

```
{
```

```
    [Obsolete("Use NewCalculate() instead")]
```

```
    static int OldCalculate(int a, int b) => a + b;
```

```
    [Author("Alice", Version = 2)] // positional + named parameter
```

```
    static void NewCalculate() => Console.WriteLine("New calculation logic");
```

```
    static void Main()
```

```
    {
```

```
        NewCalculate();
```

```
        // Reading custom attribute via Reflection
```

```
        MethodInfo method = typeof(Program).GetMethod("NewCalculate",
```

```
            BindingFlags.NonPublic | BindingFlags.Static);
```

```
        var attr = (AuthorAttribute)Attribute.GetCustomAttribute(method,
```

```
            typeof(AuthorAttribute));
```

```
        if (attr != null)
```

```

        Console.WriteLine($"Author: {attr.Name}, Version: {attr.Version}");
    }
}

```

Typical Assignment Program

Problem: Create a custom attribute [Reviewed(ReviewerName, Date = "...")] that can be applied to methods. Apply it to two methods in a class, then use Reflection to list all reviewed methods along with the reviewer's name.

```
using System;
```

```
using System.Reflection;
```

```
[AttributeUsage(AttributeTargets.Method, AllowMultiple = false)]
```

```
class ReviewedAttribute : Attribute
```

```

{
    public string ReviewerName;
    public string Date { get; set; }

    public ReviewedAttribute(string reviewerName)
    {
        ReviewerName = reviewerName;
    }
}

```

```
class Calculator
```

```

{
    [Reviewed("John", Date = "2026-01-15")]
    public int Add(int a, int b) => a + b;

    [Reviewed("Sara", Date = "2026-02-10")]
    public int Multiply(int a, int b) => a * b;

    public int Subtract(int a, int b) => a - b; // not reviewed
}

```

```
class Program
```

```

{
    static void Main()
    {
        Type type = typeof(Calculator);

        foreach (MethodInfo method in type.GetMethods())
    }
}

```

```

    {
        var attr = (ReviewedAttribute)Attribute.GetCustomAttribute(method,
typeof(ReviewedAttribute));
        if (attr != null)
            Console.WriteLine($"{method.Name} reviewed by {attr.ReviewerName} on
{attr.Date}");
    }
}
}

```

22. Asynchronous Programming

Description

Asynchronous programming lets a program start long-running operations (I/O, network, file access) without blocking the calling thread, improving responsiveness (UI apps) and scalability (server apps). C# implements this cleanly using the async/await pattern built on Task.

Syntax

```

async Task MethodNameAsync()
{
    await SomeAsyncOperation();
}

```

```

async Task<ReturnType> MethodNameAsync()
{
    ReturnType result = await SomeAsyncOperation();
    return result;
}

```

Types

Type	Description
async void	Used only for event handlers; cannot be awaited, exceptions not easily caught
async Task	Represents an asynchronous operation with no return value (awaitable)
async Task<T>	Represents an asynchronous operation that returns a value of type T
Task.Run()	Offloads CPU-bound work to a background thread
Task.WhenAll() / Task.WhenAny()	Run multiple tasks concurrently and wait for all/any to complete

Use

- Keep UI responsive while performing long operations (file download, database query)
- Improve server throughput by freeing threads during I/O waits instead of blocking them

- Run multiple independent operations concurrently (e.g., calling several APIs at once)

Full Example

```
using System;
```

```
using System.Threading.Tasks;
```

```
using System.Net.Http;
```

```
class AsyncDemo
```

```
{
    static async Task<string> DownloadDataAsync(string url)
    {
        using (HttpClient client = new HttpClient())
        {
            Console.WriteLine("Download started...");
            string result = await client.GetStringAsync(url);
            Console.WriteLine("Download completed.");
            return result;
        }
    }

    static async Task SimulateWorkAsync(string name, int delayMs)
    {
        Console.WriteLine($"{name} started");
        await Task.Delay(delayMs); // simulate async I/O wait
        Console.WriteLine($"{name} finished after {delayMs}ms");
    }

    static async Task Main()
    {
        Console.WriteLine("Main started");

        // Run multiple async tasks concurrently
        Task t1 = SimulateWorkAsync("Task A", 2000);
        Task t2 = SimulateWorkAsync("Task B", 1000);
        await Task.WhenAll(t1, t2);

        Console.WriteLine("All tasks completed");
        Console.WriteLine("Main finished");
    }
}
```

Typical Assignment Program

Problem: Simulate a food delivery app: create async methods `PrepareFoodAsync()` (3 seconds) and `ArrangeDeliveryAsync()` (2 seconds) that run concurrently using `Task.WhenAll`, then a final method `DeliverOrderAsync()` that runs only after both complete. Use try-catch to handle any exception during the process.

```
using System;
```

```
using System.Threading.Tasks;
```

```
class FoodDeliveryDemo
```

```
{
```

```
    static async Task PrepareFoodAsync()
```

```
    {
```

```
        Console.WriteLine("Preparing food...");
```

```
        await Task.Delay(3000);
```

```
        Console.WriteLine("Food is ready.");
```

```
    }
```

```
    static async Task ArrangeDeliveryAsync()
```

```
    {
```

```
        Console.WriteLine("Arranging delivery rider...");
```

```
        await Task.Delay(2000);
```

```
        Console.WriteLine("Rider is ready.");
```

```
    }
```

```
    static async Task DeliverOrderAsync()
```

```
    {
```

```
        Console.WriteLine("Both food and rider are ready. Delivering order...");
```

```
        await Task.Delay(1000);
```

```
        Console.WriteLine("Order delivered successfully!");
```

```
    }
```

```
    static async Task Main()
```

```
    {
```

```
        try
```

```
        {
```

```
            Console.WriteLine("Order placed.");
```

```
            Task foodTask = PrepareFoodAsync();
```

```
            Task riderTask = ArrangeDeliveryAsync();
```

```
            await Task.WhenAll(foodTask, riderTask); // wait for both concurrently
```

```

        await DeliverOrderAsync(); // runs after both are done
    }
    catch (Exception ex)
    {
        Console.WriteLine("Error during delivery process: " + ex.Message);
    }
}
}

```

Project on:

Scenario: A Smart Campus Management System — one connected story where Students and Instructors (inheriting from an abstract Person) get enrolled in courses, pay fees/get paid via a shared IPayable interface, receive grades, get notified by email/SMS asynchronously, and generate LINQ-based GPA reports — all in a single flowing Program.cs, not a menu of unrelated demos.

Every topic is load-bearing in the story, e.g.:

- Enrollment hits capacity → student goes into a Queue waiting list (Collections)
- Re-enrolling the same student → throws a custom EnrollmentException (Try/Catch)
- Student is split across two files as a **partial class** (core identity vs academic record)
- Person.ShowId() is overridden by Introduce() but *hidden* by ShowId() in Student/Instructor — shown side-by-side so the difference is visible
- A custom [ModuleInfo(...)] attribute documents services and is read back via reflection at startup

End of Unit 1 Detailed Notes — Language Preliminaries